LeNet 加速器微架構說明書

Microarchitecture Specification　版本 2.0　/　V9 RTL

五層 LeNet 共用一組 25 個 signed 8-bit 乘法器。CONV1、CONV2 計算空間視窗，CONV3、FC1、FC2 分組計算向量內積。指定 Golden 輸入在 V9 執行 29,481 cycles。第 14 ~ 16 章記錄群組選擇、逐版量測與合成時序條件。

1 架構與資料流總覽

層	Dataflow 資料流	資料如何重用
CONV1	Row-Stationary-inspired + Weight Stationary	六列輸入暫存重用相鄰視窗；25 個權重固定在本地；只有一個輸入通道。
CONV2	Row-Stationary-inspired + Weight Stationary	沿用列重用與固定權重；2×10×32-bit 部分和累加六個輸入通道。
CONV3	Output Stationary（OS）	400 筆輸入分為 50 個候選組，各輸出累加已核發的組。
FC1	Output Stationary（OS）	120 筆輸入分為 15 個候選組，以 bank 非零旗標選組。
FC2	Output Stationary（OS）	累加 84 個乘積：10 組各 8 筆，最後一組 4 筆。

Stationary 指資料在一段計算期間留在本地：Weight Stationary 留住目前 kernel 的權重；Output Stationary 留住目前輸出的部分和位置，數值仍持續更新。Row-Stationary-inspired 表示採用列資料重用的想法，不宣稱完整實作某種典型陣列。

資料流術語

act：輸入值；weight：權重；row_sum：一列的加總；channel_sum：一個輸入通道的加總；conv_out：所有輸入通道加完的卷積值；psum：尚未加完的部分和。PE 是處理單元，每個含 5 個乘法器。SRAM 是儲存輸入、權重與輸出的記憶體。

Spatial / Vector 是硬體運算模式，dataflow 是資料保留與搬移方式，兩者描述不同面向。FC buffer 維持 120 bytes；FC1 完成後載回 84 筆給 FC2，不增加另一組 84-byte 暫存區。

2　頂層架構與共用資料路徑

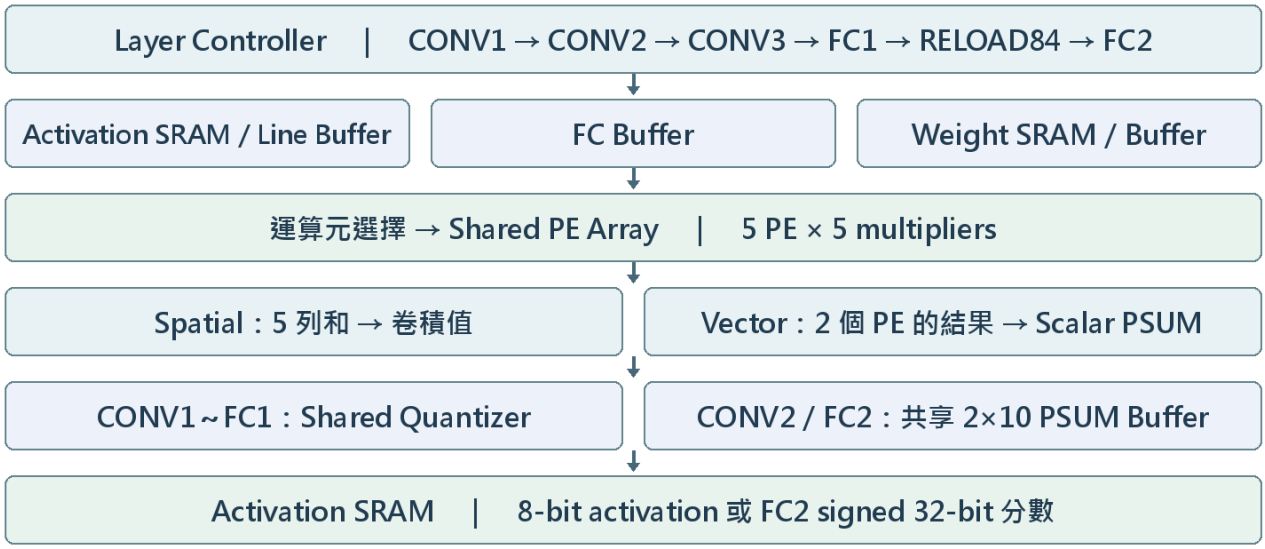


圖 1　頂層依作用層選擇運算元、累加目的地與輸出路徑。各層後處理順序見第 11 章。

Spatial 模式將 5×5 視窗與 kernel 送入五個 PE。Vector 模式取 PE0、PE1 各四個乘法位置，其他位置輸入補零。CONV3 借用共用 Line Buffer 首列的前 8 bytes 暫存 SRAM 回傳值。

CONV1、CONV2、CONV3、FC1 分時使用同一個 Shared Quantizer。FC2 讀取共享 PSUM Buffer 的含 bias 結果，以 32-bit word 寫回。

2.1 Lenet Function Block Diagram

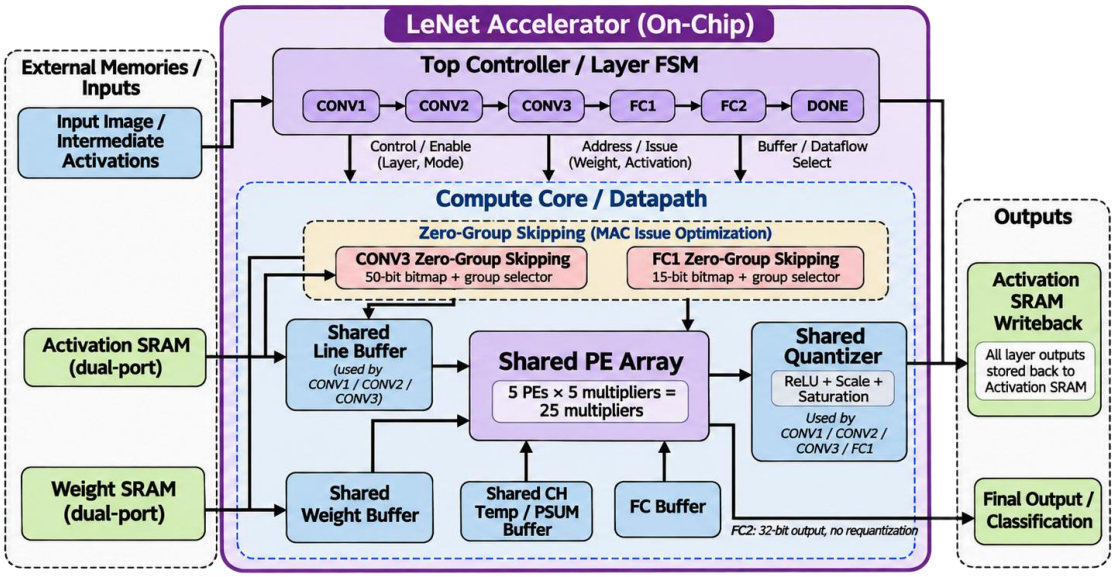


圖 1A　Lenet 功能方塊圖。箭頭表示主要資料或控制關係，實際多工與時序請參照 RTL。

頂層 FSM 依層選擇 SRAM、運算元與共用資源。CONV3 與 FC1 的 Group Selector 由各自的非零資訊選出群組。Shared PSUM Buffer 由 CONV2 與 FC2 分時使用。圖中 SRAM 位於 Lenet 頂層外部。

3 層級控制與位置編號

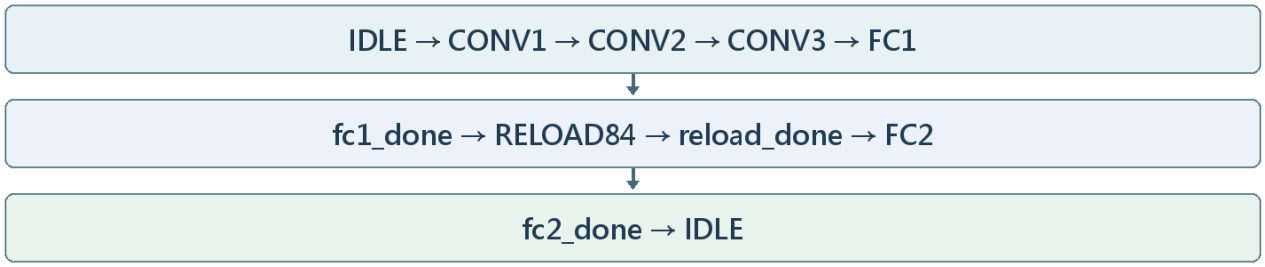


圖 2　頂層 FSM 的層級順序。每次完成推論後返回 IDLE。

兩列區塊與輸出座標

strip 是同一輸出通道中的「相鄰兩列輸出區塊」；strip_idx 是區塊編號。row_sel 選區塊內的列：0 是上列、1 是下列；col 是 column（欄位）。idx 是 index（編號），sel 是 select（選擇），所有編號從 0 開始。

位置名稱	CONV1	CONV2	用途
out_ch	0 ~ 5	0 ~ 15	輸出通道編號
in_ch	0	0 ~ 5	輸入通道編號
strip_idx	0 ~ 13	0 ~ 4	第幾個兩列區塊
row_sel	0 或 1	0 或 1	區塊內上列或下列
col	0 ~ 27	0 ~ 9	pooling 前的輸出欄位

例如 strip_idx=3、row_sel=1，是整張輸出的第 7 列：out_row=2×strip_idx+row_sel=7。Vector 則用 neuron_idx（神經元編號）和 group_idx（輸入組別）計數。

fc1_done 啟動頂層 RELOAD84 狀態。reload_done 表示末筆載回完成，此後 FC2 才能使用 FC Buffer。fc2_done 使狀態返回 IDLE，完成訊號由頂層產生。

4 乘法核心與三層加總

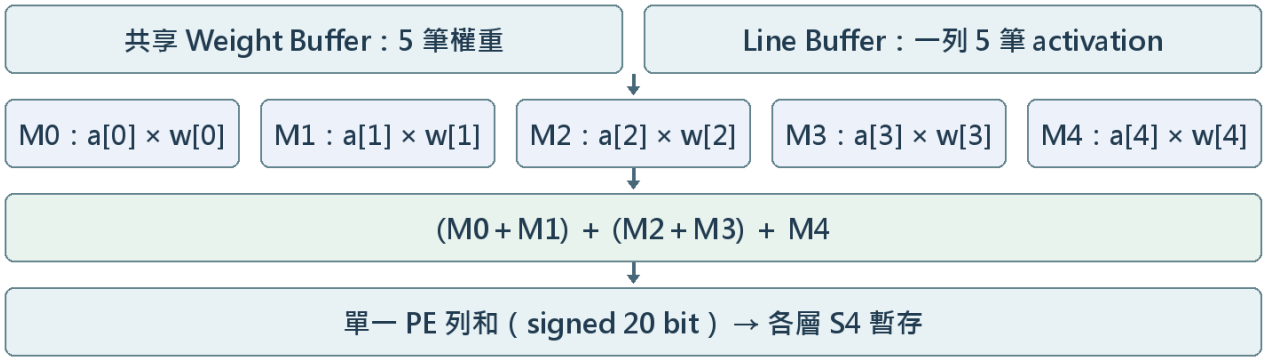


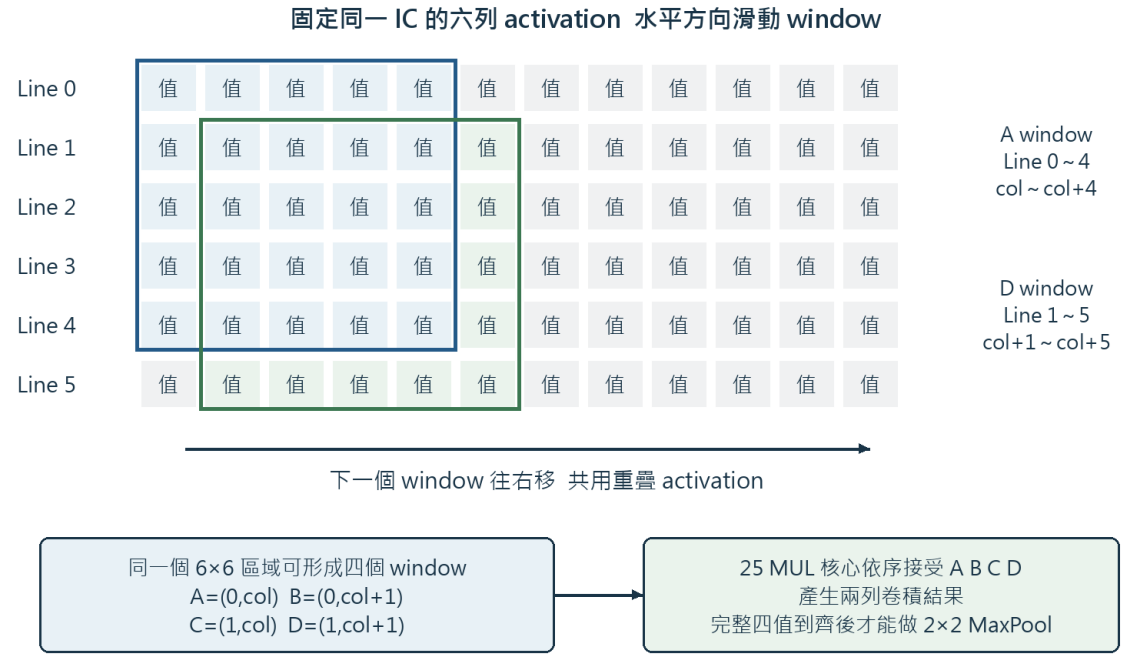
圖 3　單一 PE 在 Spatial 模式計算五個乘積。右側的列和介面依 V9 頂層參數配置為 20 bit。

權重來自 PE 外部的共享 Weight Buffer。每對 8-bit 有號運算元相乘，五個乘積在組合加法樹中歸約。18 bit 足以表示五項列和，V9 實體輸出為 signed 20 bit。各層將 PE 結果暫存至 S4 後再合併。

步驟	做什麼	數字例子
row_sum 列加總	同一列五個輸入各乘對應權重，再相加。	輸入 [1,2,3,4,5]、權重 [2,-1,0,1,3] → 2-2+0+4+15=19。
channel_sum 單一通道加總	把五列的 row_sum 相加，得到一個輸入通道的 25 個乘積和。	五列 [19,5,-2,8,10] → 40。
conv_out 完整卷積結果	把各輸入通道的 channel_sum 相加。	Conv2 六通道 [40,10,-5,20,0,15] → 80。

表中數字用來說明累加層級。CONV1 的一個 channel_sum 就是完整卷積值；CONV2 需累加六個輸入通道，最後再進入 pooling。

5 六列輸入暫存與兩列區塊排程



本圖名詞：Line = 輸入列；方格內的「值」代表一筆輸入數值；window = 卷積視窗
A、B、C、D 僅是四個視窗的名稱：上左、上右、下左、下右，不是運算結果變數
IC = Input Channel 輸入通道；col = Column 起始欄位編號；(列,欄) 表示視窗起點
MUL = Multiplier 乘法器；MaxPool = 區域取最大值；reuse = 重複使用資料

圖 4 上列輸出需輸入第 0～4 列，下列需第 1～5 列；合起來只需保留六列。

5×5 視窗向右移一欄會重用四欄，向下移一列會重用四列。這就是 Row-Stationary-inspired 的輸入重用；同一組 25 個權重則留在 Weight Reg，形成 Weight Stationary。

排程順序：固定輸出通道 → 選定兩列區塊 → 讓一個輸入通道掃完整個區塊 → 換下一個輸入通道。換通道時載入對應輸入與權重；現有六列暫存不代表同時保存所有輸入通道。

實體容量為 6 列 × 8 個 32-bit 區塊，共 192 bytes。CONV1 在同一欄先算上列再算下列，完成後向右移。下一個 strip 沿用四列、補入兩列。CONV2 切換輸入通道時重新載入六列。

6 CONV1 詳解

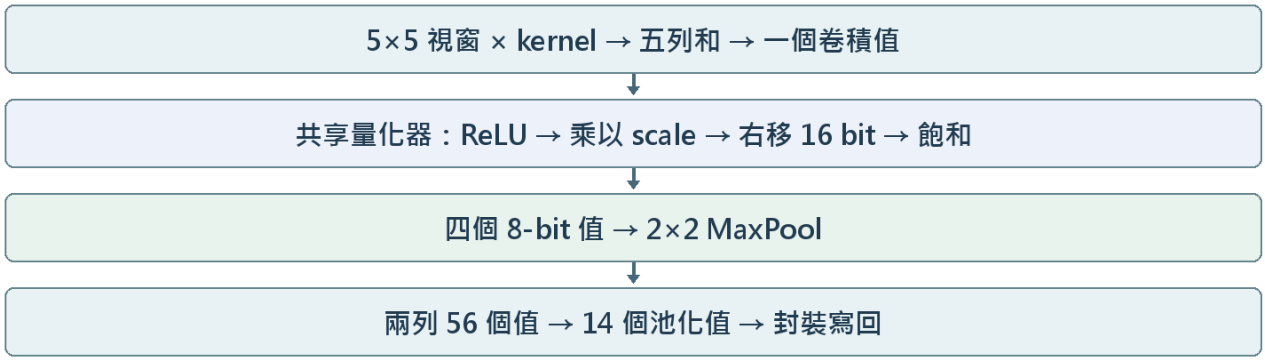


圖 5　CONV1 的每個完整卷積值先量化，2×2 比較器接收 8-bit 量化結果。

資料流與計算

CONV1 採 Row-Stationary-inspired + Weight Stationary：六列暫存重用重疊輸入，25 個權重暫存器保留目前輸出通道的 5×5 kernel，供多個位置反覆使用。換輸出通道時才換對應權重；載入仍需時間。

輸入為 1×32×32，卷積輸出為 6×28×28。每個位置只需一次 25 個乘積的加總，因為輸入通道數只有 1，不需跨通道 PSUM。

Pooling 與寫回

Streaming MaxPool 表示結果陸續產生、陸續交給 pooling，不必先存完整特徵圖。但同一個 2×2 區域仍要收齊四個值，才能選最大值；尚未配對的值需要暫存。

每個 strip 有 2×28 個卷積值。每個值先做 ReLU、定點縮放及飽和，再由 2×2 MaxPool 產生 14 個輸出。各通道完成後，寫回尺寸為 6×14×14。

本層檢查重點

確認視窗邊界與 pooling 四值位置正確；不寫跨通道 PSUM；換輸出通道前，尚未使用完的輸入或結果不得被覆蓋。

7 CONV2 詳解

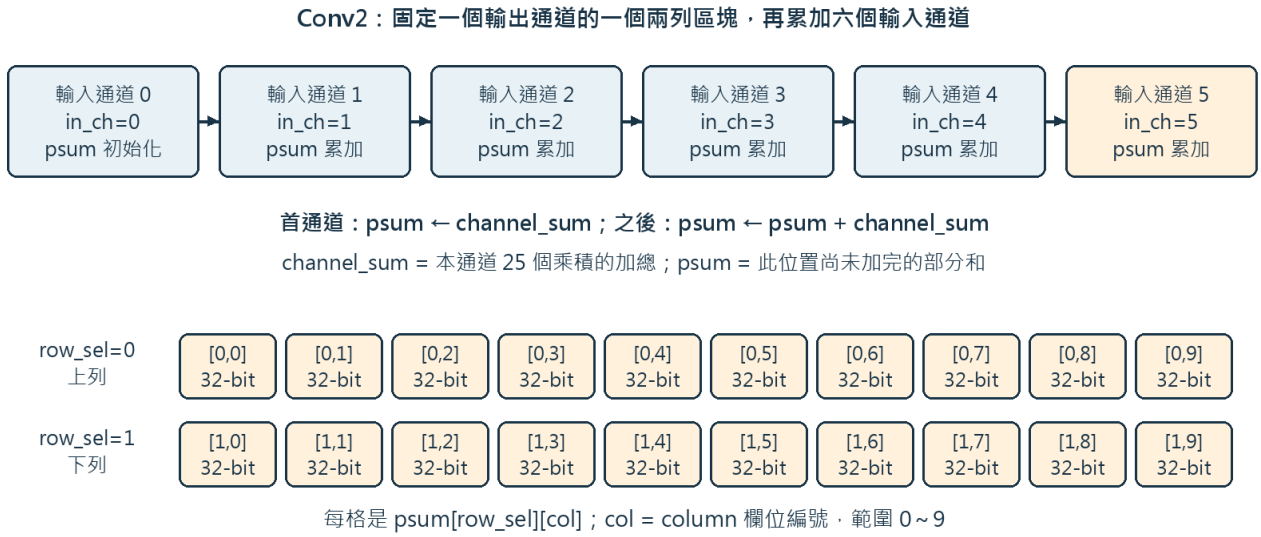


圖 6 每格保存一個位置的部分和；六個輸入通道全部加完後才做 pooling。

CONV2 同樣採 Row-Stationary-inspired + Weight Stationary。每個位置需累加六個輸入通道；目前通道的 25 個權重在掃描區塊期間固定，換通道時更新，並非六通道共用同一組權重。

共享 PSUM Buffer 有 2×10 個 signed 32-bit 位置，共 80 bytes。第一個輸入通道覆寫對應格，後續五個通道讀取舊值、累加並同步寫回。六個通道共用同一組位置。

16×10×10 個完整卷積值以兩列為單位處理。最後一格 PSUM 寫入後，連續五次從相鄰兩欄組合讀出四格，做 signed 32-bit MaxPool，再經共享 ReLU / Quantizer 輸出 16×5×5 個 bytes。

8 四級管線與有效資料

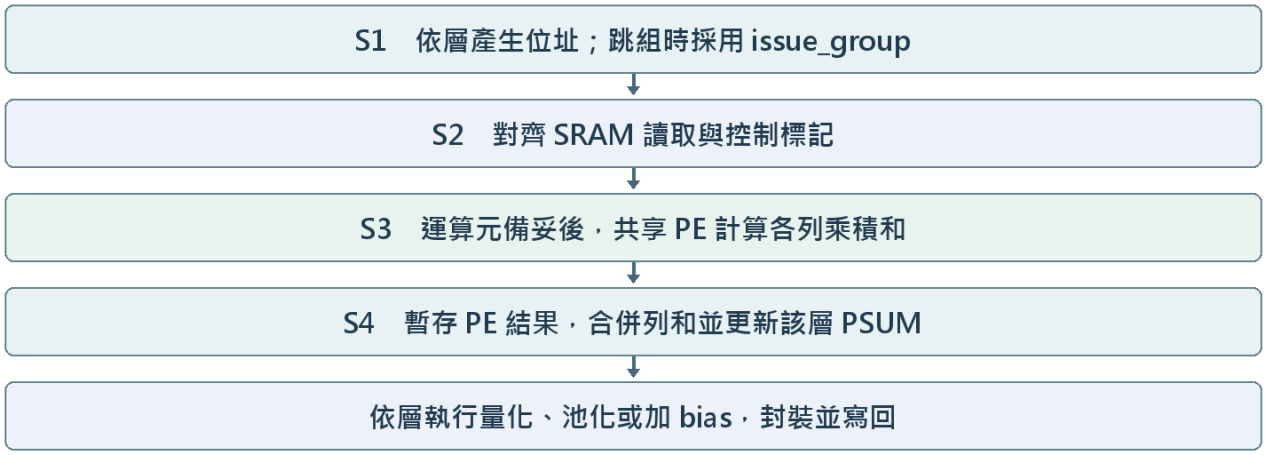


圖 7　位址、回傳與 PE 運算依時脈對齊。S4 暫存 PE 結果，提供後級歸約、累加及寫回。

階段	本級工作
S1 Address Gen 發送位址	產生 SRAM 位址或本地 buffer 索引；跳組層用實際選出的原始組號。
S2 Data Fetch & Buffer 接收並暫存	依固定延遲對齊 SRAM 資料、有效標記與索引。
S3 Execute 乘加運算	Spatial 取 25 對運算元；Vector 取 8 對，尾組依層遮蔽。PE 內以加法樹歸約。
S4 Writeback 後處理與寫回	暫存 PE 輸出後合併列和、累加部分和；完成後依層進入後處理及寫回。

跳組時需延遲傳遞實際核發的 issue_group。請求與回傳資料依各層 valid 管線配對，PE 結果暫存後再依對齊的 group index 判斷初始化或完成。此圖為階段示意，未按量測週期比例繪製。

9 CONV3 FC1 FC2 的向量運算

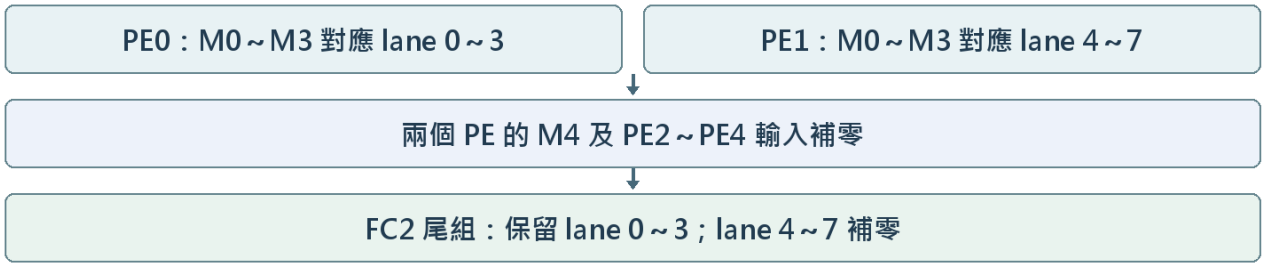


圖 8　向量模式的實體 lane 配置。一般組使用 4＋4 lanes，FC2 尾組只使用低四個位置。

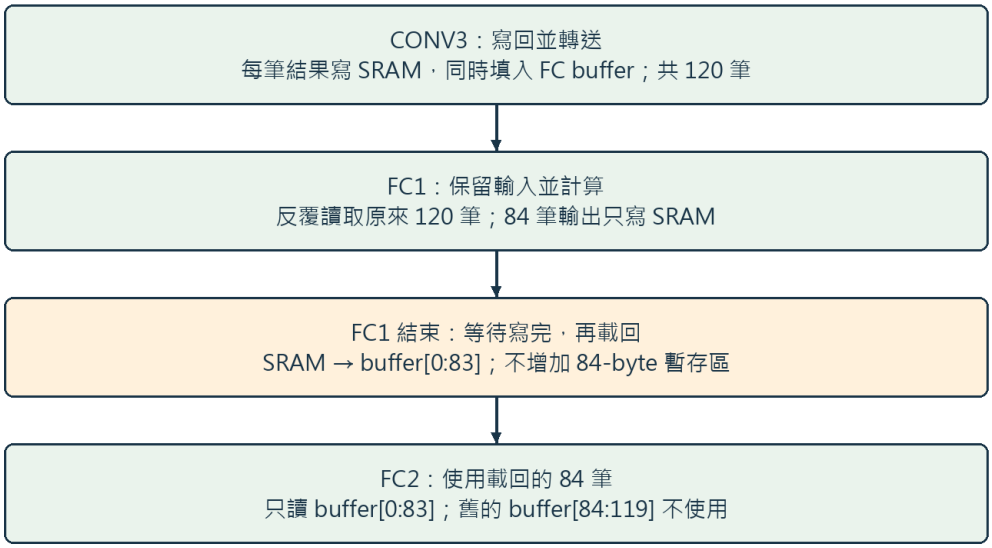
三層皆採 Output Stationary（OS）：本組乘積先相加成 group_sum，再加進同一個 scalar_psum，直到目前神經元完成。第一組初始化部分和，後續接續最新值；不是每個輸出各配置一個累加器。

層	每個輸出的稠密工作量	輸入來源與完成結果
CONV3	400 筆 = 50 組 × 8	Activation SRAM 提供 16×5×5；完成 120 筆輸出後，寫 SRAM 並轉送 FC buffer。
FC1	120 筆 = 15 組 × 8	反覆讀 FC buffer 的 120 筆；完成 84 筆輸出，只寫 SRAM。
FC2	84 筆 = 10 組 × 8 + 4	讀 FC Buffer 前 84 筆；輸出十個含 bias 的 signed 32-bit 分數。

FC2 每個神經元有 11 組。第 10 組（從 0 編號）只取輸入 80～83；activation 與 weight 的高 32 bit 均補零，PE1 在這一組的貢獻為零。

CONV3、FC1 保留原始組號，從 bitmap 選出要執行的群組。FC2 依序執行各組。選擇規則及指定測試資料的核發次數見第 14 章。

10 FC Buffer 更新與供料速度



本圖名詞：CONV3 = 第 3 卷積層；FC1 / FC2 = 第 1 / 第 2 全連接層
SRAM = 靜態隨機存取記憶體；FC buffer = 全連接層輸入暫存區
buffer[0:83] = 編號 0～83 的 84 格；buffer[84:119] = 編號 84～119 的舊資料格

圖 9 保留單一 120-byte FC buffer；FC1 計算期間不覆寫原輸入。

FC1 的 84 個輸出共用同一份 120-byte 輸入。若提前覆寫，後續輸出可能使用錯誤 activation。因此 FC1 先將結果寫入 SRAM，整層結束後再載回 buffer[0:83]。

RELOAD84 透過兩個 32-bit SRAM 埠讀取 11 組。前十組各載入 8 bytes，末組寫入位址 80，僅啟用低四個 bytes（wstrb = 0x0F）。這是傳輸組數，總週期另含位址與資料管線。

FC Buffer 分為 15 個 64-bit bank。每個 bank 的 reduction OR 形成一個非零旗標，供 FC1 Group Selector 判斷該組是否需要取權重。

11 各層硬體分工與工作量

層與尺寸	模式與乘法器	輸入與權重來源
CONV1 1×32×32 → 6×28×28	Spatial；25 個	Activation SRAM → 六列暫存；25 Weight Reg 重用目前 kernel。
CONV2 6×14×14 → 16×10×10	Spatial；25 個	Activation SRAM → 六列暫存；換輸入通道時換 kernel。
CONV3 16×5×5 → 120	Vector；8 個	輸入與權重均由 SRAM 供應；每個輸出重新讀其所需輸入。
FC1 120 → 84	Vector；8 個	FC buffer[0:119] 提供輸入；Weight SRAM 串流。
FC2 84 → 10	Vector；8 個，尾組 4	FC buffer[0:83] 提供輸入；Weight SRAM 串流。

層	PSUM 與輸出處理	稠密工作量
CONV1	卷積 → ReLU / Quantize → MaxPool → 寫回。	4,704 個視窗；117,600 次乘法。
CONV2	六通道累加 → MaxPool → ReLU / Quantize → 寫回。	9,600 個單通道視窗；240,000 次乘法。
CONV3	Scalar PSUM → ReLU / Quantize → 寫回、轉送。	6,000 組；48,000 次乘法。
FC1	Scalar PSUM → ReLU / Quantize → 寫回；整層完成後 RELOAD84。	1,260 組；10,080 次乘法。
FC2	Scalar PSUM → 加 bias → signed 32-bit 寫回。	110 組；840 次乘法。

上表工作量依完整網路形狀計算，共 416,520 次乘法。Spatial 統計單通道視窗，Vector 統計 8-lane group；第 14 章另列實際核發數。端到端 cycles 還包含載入、管線填充與排空、pooling、寫回及 RELOAD84。

12 寫回 數值與索引規則

寫回介面

各層產生 SRAM 位址、資料與逐 byte 寫入致能，由頂層依 FSM 狀態選擇埠訊號。固定延遲由 valid 管線對齊；介面未提供通用 ready/backpressure 握手。

CONV1 至 FC1 將 8-bit 結果封裝至 32-bit SRAM word。FC2 將十個加上 bias 的 signed 32-bit 分數逐字寫到 Activation SRAM 的 word 位址 743 ~ 752。

數值處理

加總範圍	最低帶符號位寬	本版處理
單一 8×8-bit 乘積	16-bit	乘法兩端均解讀為 signed。
一列 5 個乘積	18-bit	理論最低 18 bit；V9 PE 輸出為 20 bit。
一個視窗 25 個乘積	20-bit	不可先截成 16-bit 再加。
CONV2 的 150 個乘積	23-bit	Strip PSUM 使用 32-bit。
CONV3 的 400 個乘積	24-bit	Scalar PSUM 使用 32-bit。

共享量化器接收 signed 32-bit PSUM 與該層 unsigned 32-bit scale。先將非正值設為零，再以 64-bit 乘積右移 16 bit，捨去低位；超過 127 時飽和為 127。FC2 的 signed 32-bit 結果保留負值。

向量與權重索引

組內第 l 筆 activation 的線性位置為 8g + l，其中 g 是實際選中的原始組號。例如 g=2、l=3 對應第 19 筆。跳組後仍以原始 g 定位對應權重。

CONV3 展平：act_idx=25×in_ch+5×in_row+in_col。例：通道 2、列 1、欄 3 → 50+5+3=58。輸入與權重要採同一順序。FC2 每個神經元 84 bytes，32-bit word 對齊但不一定在 8-byte 邊界；讀取電路須選正確 word，尾組只用四筆。

13　共用儲存與驗證範圍

儲存	容量	用途
Weight Reg	25×8-bit = 25 bytes	Spatial 的目前 kernel。
Shared PSUM	2×10×32-bit = 80 bytes	CONV2 strip；FC2 內積與含 bias 結果。
Scalar PSUM	每層 1×32-bit	Vector 各層控制器的累加狀態。
FC_Act_Buffer	120×8-bit = 120 bytes	FC1 用 120 格；FC2 用前 84 格。
6-Line Buffer	6×32 bytes = 192 bytes	Spatial 六列視窗；CONV3 借用首列 8 bytes。

表中列出邏輯儲存用途。各 Vector 層在自身控制模組保存 scalar PSUM；合成後的資源用量以 DC Total Cell Area 為準。

CONV2 與 FC2 的 PSUM Buffer

CONV2 用兩列保存同一 strip 的 20 個部分和。切換到 FC2 後，row 0 改存十個輸出的最終內積；每次讀入一對 bias，將偶、奇輸出各自加上 bias 後寫至 row 1，再由 row 1 讀出寫回 SRAM。

驗證依據

指定 Golden 案例的 RTL 比對與 gate-level simulation 均回報 PASS。此結果涵蓋該輸入的五層輸出與完整寫回流程。

跳組控制保留首尾組，維持第一組初始化與尾組完成的條件。FC2 遮蔽尾組多餘四筆，RELOAD84 使用尾組 byte mask。

合成階段的 10 ns setup 結果見第 16 章。現有紀錄的範圍為指定測試資料與合成條件；多輸入回歸、post-route STA 與完整 PVT signoff 尚無結果。

功能依據：hdl/lenet.sv、各層控制器、shared_pe_array.sv、shared_line_buffer.sv、shared_ch_temp_buffer.sv、shared_quantizer.sv、fc_buffer.sv 與 reload_controller.sv。

14　Zero-Group Skipping 與實際請求

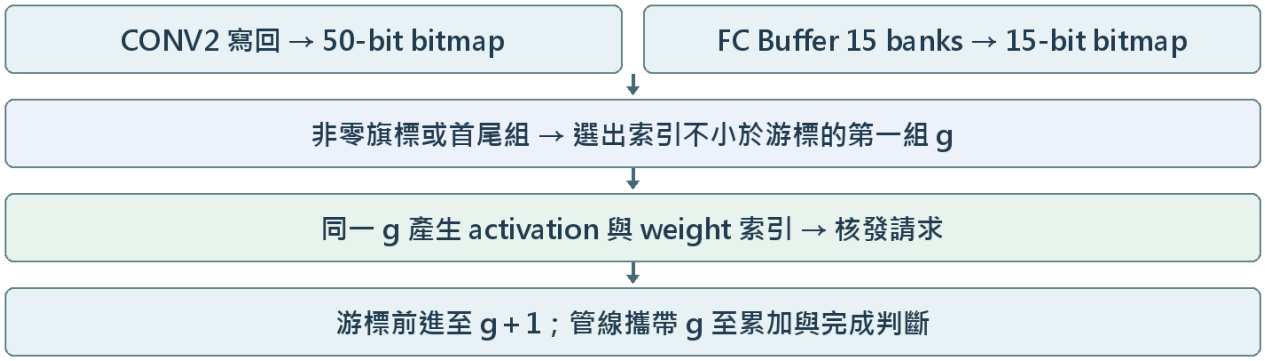


圖 10　CONV3 與 FC1 各有選組邏輯。兩層共用相同的核發原則。

CONV3 的 bitmap 在 reset 或 compute_start 清零。CONV2 寫回 100 個完整的 32-bit words 時，相鄰兩個 word 對應一組；任一 word 非零便將該組設為 1，並保留至本次推論結束。FC1 直接對每個 64-bit bank 做 reduction OR。兩種 bitmap 都以 1 表示該組含非零 activation。

CONV3 強制保留組 0、49；FC1 以 0x4001 保留組 0、14。即使首尾組全零也會核發，讓首組初始化與末組輸出沿用原有條件。

層	輸出數	每輸出組數 稠密 → Golden	稠密請求	實際請求	省略請求
CONV3	120	50 → 36	6,000	4,320	1,680
FC1	84	15 → 11	1,260	924	336
FC2	10	11 → 11	110	110	0
合計	—	—	7,370	5,354	2,016

一筆 request 對應一個向量組。此 Golden 的核發數由每輸出保留組數乘以輸出數取得；activation 的零值分布隨輸入改變。